



网络安全创新与创业实践

SM3 软件实现与优化

耿介然 202300460096 网安 23.1

2026 年 7 月 29 日

目录

1	实验目的	1
2	实验原理	1
2.1	SM3 算法概述	1
2.2	优化思路: SIMD 与通用寄存器混合	1
3	实验内容	1
3.1	基础实现 (sm3.c)	1
3.2	AVX2 优化 (sm3_avx2.c)	2
3.3	NEON 优化 (sm3_neon.c)	2
3.4	编译与构建	2
4	实验结果	3
4.1	正确性验证	3
4.2	性能对比	3
5	实验小结	3

1 实验目的

本次实验的主要目的是理解国密 SM3 杂凑算法的工作原理，并用 C 语言完成其软件实现。在正确实现的基础上，进一步尝试利用 CPU 的单指令多数据（SIMD）扩展——x86 平台的 AVX2 与 ARM64 平台的 NEON——对算法进行性能优化，并对比优化前后的吞吐量差异。

2 实验原理

2.1 SM3 算法概述

SM3 是我国商用密码杂凑算法标准（GB/T 32905-2016），它接受任意长度的输入消息，输出 256 位（32 字节）的摘要值。算法内部以 512 位（64 字节）为一个分组对消息进行处理，每个分组的处理分为两个阶段：

1. **消息扩展：**将当前 64 字节分组的 16 个字（每字 32 位）扩展为 132 个字 $W_0, W_1, \dots, W_{67}$ 和 $W'_0, W'_1, \dots, W'_{63}$ ，供后续压缩使用。
2. **压缩函数：**用扩展后的 W 值对 256 位的链接变量 A, B, C, D, E, F, G, H 进行 64 轮迭代压缩，每轮更新全部 8 个变量。

最后对末尾不足一组的数据进行填充、写入消息长度，逐组处理后输出 256 位摘要。

2.2 优化思路：SIMD 与通用寄存器混合

消息扩展阶段中存在大量“互不依赖”的异或和循环移位运算。例如扩展公式中 $W_{j-16} \oplus W_{j-9}$ 和 $(W_{j-13} \lll 7) \oplus W_{j-6}$ 这两项，当 j 取连续 4 个值时，它们的数据彼此独立，可以用 SIMD 指令一次并行计算 4 路。

但 W_j 的递推还包含一项 $P_1(W_{j-3} \lll 15)$ ，它依赖前面刚算出的结果，无法并行——这条依赖链只能由通用寄存器（GPR）串行完成。

压缩函数中的 64 轮迭代是更强的串行依赖链：每一轮的输出立即成为下一轮的输入，无法拆成独立子任务，强行用 SIMD 只会增加数据在 SIMD 寄存器与通用寄存器之间来回搬运的开销。因此压缩阶段保持 GPR 串行推进。

阶段	并行特征	采用方式
消息扩展	部分大项彼此独立，可 4 路并行	SIMD（AVX2 YMM / NEON）
压缩函数	强数据依赖链，不可拆分	GPR 串行

这种“扩展用 SIMD、压缩用 GPR”的混合策略，是 SM3 在通用 CPU 上最现实的优化路径。

3 实验内容

3.1 基础实现（sm3.c）

首先按照国标 GB/T 32905-2016 的算法描述，用纯 C 语言（仅使用通用寄存器）完成 SM3 的完整实现，包括：

- 置换函数 P_0 、 P_1 ;
- 布尔函数 FF_j 、 GG_j (前半轮与后半轮不同);
- 消息扩展函数 `sm3_expand`;
- 压缩函数 `sm3_compress`;
- 流式 API: `sm3_init`、`sm3_update`、`sm3_final`。

基础实现作为正确性基准, 所有优化版本的结果都与其比对。

3.2 AVX2 优化 (`sm3_avx2.c`)

在消息扩展函数中用 Intel AVX2 的 256 位 YMM 寄存器, 每次载入连续的 4 个 32 位字 (共 128 位未用满), 执行并行异或与循环左移:

- `_mm256_xor_si256` 一次完成 4 路异或;
- `_mm256_slli_epi32` / `_mm256_srli_epi32` 组合实现 4 路并行循环左移。

计算得到的外部异或项存入数组后, 再用 GPR 串行补上 P_1 依赖项, 合并得到最终的 W_j 。压缩函数部分保持与基础实现一致的纯 GPR 结构。

3.3 NEON 优化 (`sm3_neon.c`)

采用与 AVX2 版本完全相同的混合策略, 在 ARM64 NEON 上用 128 位 `uint32x4_t` 寄存器做 4 路并行:

- `veorq_u32` 完成并行异或;
- `vshlq_n_u32` / `vshrq_n_u32` / `vorrq_u32` 组合实现并行循环左移。

在非 ARM64 平台上, `sm3_neon.c` 退化为调用基础 `sm3_hash` 的占位实现, 以保证整个工程可链接、可运行。

3.4 编译与构建

使用 Makefile 统一管理编译流程:

- `make`: 编译生成可执行文件 `sm3_test`;
- `make test`: 运行正确性测试;
- `make bench`: 运行性能基准测试。

Makefile 会自动检测 CPU 是否支持 AVX2, 若支持则添加 `-mavx2` 编译选项。

4 实验结果

4.1 正确性验证

使用 GB/T 32905-2016 附录中的标准测试向量进行验证，三个实现（基础版、AVX2 版、NEON 版）的测试结果如下：

表 1: 标准测试向量验证结果

测试用例	基础版	AVX2 版	NEON 版
"abc" (3 字节)	PASS	PASS	PASS
"abcd"×16 (64 字节)	PASS	PASS	PASS
流式分段一致性 (update 分段调用)	PASS	—	—

所有测试向量均通过，说明优化后的实现与基础实现输出完全一致，优化并未破坏算法的正确性。

4.2 性能对比

在支持 AVX2 的 x86-64 平台上，对 64 KB 数据重复哈希 2000 次，测得吞吐量如下：

表 2: 性能对比 (64 KB 数据 × 2000 次)

实现版本	吞吐量	相对提升
基础实现	~227 MB/s	—
AVX2 优化	~267 MB/s	+17.6%

消息扩展阶段经过 SIMD 化后带来了约 17% 的整体吞吐提升。由于压缩函数占计算时间的大头且无法向量化，提升幅度受限于扩展阶段的占比。

5 实验小结

本次实验完成了以下工作：

1. 依照国标实现了完整的 SM3 杂凑算法（含流式接口）；
2. 针对 SM3 消息扩展阶段”部分可并行”的特点，在 x86 AVX2 和 ARM64 NEON 两种平台上分别用 SIMD 指令加速了扩展运算；
3. 对压缩函数保持 GPR 串行实现，避免不必要的 SIMD/GPR 数据搬移开销；
4. 用标准测试向量验证了三个版本的正确性，并完成了性能基准对比。

通过这次实验，加深了对国密 SM3 算法结构以及 SIMD 优化适用场景的理解——并非所有计算都适合向量化，需要具体分析算法的依赖关系，选择”能并行的并行，不能并行的保持串行”的混合策略。